

c0a2515415 / ProjExD_Group07

🔍

👤

+

🕒

🔗

📄

📁

<> Code

🕒 Issues

🔗 Pull requests 1

🕒 Actions

📁 Projects

📖 Wiki

🛡️ Security and quality

📈 Insights

ProjExD_Group07 / README.md

⋮

c0a2515415 初期状態

18ca610 · 4 hours ago

🕒

31 lines (25 loc) · 1.74 KB

飛ぶな！走れ！こうかとん！

実行環境の必要条件

- python >= 3.10
- pygame >= 2.1

ゲームの概要

- 主人公キャラクター（こうかとん）をスペースキーでジャンプさせることで障害物を避けていくゲームです
- 参考URL：[サイトタイトル](#)

ProjExD_Group07 / README.md

↑ Top

PreviewCodeBlame

Raw📄📥📝⌵☰

- ステージは三つあり、最終ステージではボスが出てくるので、倒せばゲームクリアです
- ライフ5個すべて削れるとゲームオーバーです

ゲームの実装

共通基本機能

- 背景画像と主人公キャラクターの描画
- 障害物の描画と当たり判定

分担追加機能

- ボス実装（担当：多賀）：最終ステージでボスを出現させる。ボスの攻撃手段なども実装する
- 夕方ステージの障害物実装（担当：高部）：夕方ステージの障害物を実装。こうかとんが左右に動けるようにし、上からつららを落とす
- 夜ステージの障害物実装（担当：細田）：夜ステージの障害物を実装

- こうかとの必殺技実装（担当：宇田）：こうかとの下に時間経過で消える足場を出して、その上に乗れるようにする
- BGM実装（担当：清水）：ゲームのBGMを追加

メモ

- 基本的には講義内容に出てきた変数を使用しているが、新しく追加アする場合は コメントで説明を入れること
- `def __init__`では使用する変数の初期化。`def update`ではmainで都度描画する必要があるものを書いている

 c0a2515415 / ProjExD_Group07





 Code  Issues  Pull requests 1  Actions  Projects  Wiki  Security and quality  Insights

ProjExD_Group07 / rush_kokaton.py  c0a2515415 Merge branch 'main' into C0A25154/boss

f4a4222 · 1 minute ago



725 lines (589 loc) · 24.9 KB

Code Blame

Raw     

```
1  import math
2  import os
3  import random
4  import sys
5  import time
6  import pygame as pg
7
8  WIDTH = 1100 # ゲームウィンドウの幅
9  HEIGHT = 650 # ゲームウィンドウの高さ
10 GROUND = 300 # 地面の高さ
11 os.chdir(os.path.dirname(os.path.abspath(__file__)))
12
13
14  def check_bound(obj_rct: pg.Rect) -> tuple[bool, bool]:
15      """
16      オブジェクトが画面内or画面外を判定し、真理値タプルを返す関数
17      引数: こうかとかんや爆弾, ビームなどのRect
18      戻り値: 横方向, 縦方向のはみ出し判定結果 (画面内: True/画面外: False)
19      """
20      yoko, tate = True, True
21      if obj_rct.left < 0 or WIDTH < obj_rct.right:
22          yoko = False
23      if obj_rct.top < 0 or HEIGHT < obj_rct.bottom:
24          tate = False
25      return yoko, tate
26
27
28  def calc_orientation(org: pg.Rect, dst: pg.Rect) -> tuple[float, float]:
29      """
30      orgから見て, dstがどこにあるかを計算し, 方向ベクトルをタプルで返す
31      引数1 org: 爆弾SurfaceのRect
32      引数2 dst: こうかとかんSurfaceのRect
33      戻り値: orgから見たdstの方向ベクトルを表すタプル
34      """
35      x_diff, y_diff = dst.centerx-org.centerx, dst.centery-org.centery
36      norm = math.sqrt(x_diff**2+y_diff**2)
37      return x_diff/norm, y_diff/norm
38
39
40  class Bird():
41      """
42      ゲームキャラクター (こうかとかん) に関するクラス
43      動くかは未定。ボス戦の時に動けた方がいいか考えよう
44      """
```

```
45     # delta = { # 押下キーと移動量の辞書
46     #     pg.K_UP: (0, -1),
47     #     pg.K_DOWN: (0, +1),
48     #     pg.K_LEFT: (-1, 0),
49     #     pg.K_RIGHT: (+1, 0),
50     # }
51
52     def __init__(self):
53         # super().__init__() #Spriteクラス継承してるからつけてる
54         #こうかとんの画像読み込み
55         self.k_img = pg.image.load("fig/9.png")
56         self.rk_img = pg.transform.flip(self.k_img, True, False)
57         self.rect = self.rk_img.get_rect()
58
59         self.rect.x = 50
60         self.rect.y = GROUND + 140
61
62
63         #重力、ジャンプのこと↓
64         self.vy = 0 #縦方向速度
65         self.gravity = 1
66         self.jumping = False #ジャンプ中か？の判定のため。初期はジャンプしていない
67         self.jump_count = 0 # 現在のジャンプ回数
68         self.max_jump = 2 # 最大2段ジャンプ
69
70         #こうかとんの無敵技用
71
72         self.state = "normal"
73         self.hyper_life = 500
74
75     def update(self, screen, platforms):
76         key_lst = pg.key.get_pressed()
77
78         if key_lst[pg.K_LEFT]:
79             self.rect.x -= 5
80
81         if key_lst[pg.K_RIGHT]:
82             self.rect.x += 5
83
84         # 移動できる範囲を制限
85         if self.rect.left < 50:
86             self.rect.left = 50
87         if self.rect.right > 350:
88             self.rect.right = 350
89
90         self.vy += self.gravity
91         self.rect.y += self.vy
92
93
94
95         if self.rect.y >= GROUND + 140:
96             self.rect.y = GROUND + 140
97             self.vy = 0
98             self.jumping = False
99             self.jump_count = 0 # ジャンプ回数リセット
100
101
102         # 足場判定
103         for platform in platforms:
104             # 上から落ちてきたときのみ乗れる
```

```
105         if self.rect.colliderect(platform.rect):
106             if self.vy >= 0 and self.rect.bottom <= platform.rect.bottom:
107                 self.rect.bottom = platform.rect.top
108                 self.vy = 0
109                 self.jumping = False
110                 self.jump_count = 0 # 足場に乗ったらリセット
111     screen.blit(self.rk_img, self.rect)
112
113
114
115     class Map():
116     def __init__(self):
117         self.bg1_img = pg.image.load("fig/pg2_bg.png")
118         self.bg1_img_flip = pg.transform.flip(self.bg1_img, True, False)
119         self.bg2_img = pg.image.load("fig/pg4_bg.png")
120         self.bg2_img_flip = pg.transform.flip(self.bg2_img, True, False)
121         self.bg3_img = pg.image.load("fig/pg3_bg.png")
122         self.bg3_img_flip = pg.transform.flip(self.bg3_img, True, False)
123
124         self.x1 = 0 #一枚目の背景
125         self.x2 = 1672 #二枚目の背景 画像の大きさが1672だった
126
127         self.speed = 2
128
129     def update(self, screen, time):
130         """
131         時間経過によってマップを切り替える
132         """
133
134         if self.speed < 10: #背景画像の移動速度
135             self.speed += 0.01
136
137         self.x1 -= self.speed
138         self.x2 -= self.speed
139
140         #背景画像
141         if time <= 2000:
142             bg = self.bg1_img
143             bg_flip = self.bg1_img_flip
144         elif time <= 4000:
145             bg = self.bg2_img
146             bg_flip = self.bg2_img_flip
147         else:
148             bg = self.bg3_img
149             bg_flip = self.bg3_img_flip
150
151         screen.blit(bg, (self.x1, 0))
152         screen.blit(bg_flip, (self.x2, 0))
153
154         #画面外判定
155         if self.x1 <= -1672:
156             self.x1 = self.x2 + 1672
157         if self.x2 <= -1672:
158             self.x2 = self.x1 + 1672
159
160
161     class Obstacle(pg.sprite.Sprite):
162         """
163         障害物クラス
164         """
```

```
165  ✓ def __init__(self):
166      super().__init__()
167
168      rad = random.randint(20, 30) # 大小さまざまな岩が転がってくるイメージ
169      self.image = pg.Surface((2*rad, 2*rad))
170      pg.draw.circle(self.image, (211, 211, 211), (rad, rad), rad)
171      self.rect = self.image.get_rect()
172
173      self.rect.x = WIDTH
174      self.rect.bottom = GROUND + 200
175      self.vx = -random.randint(10, 20) # 障害物の移動速度
176
177  def update(self):
178      self.rect.x += self.vx
179      if self.rect.right < 0: # 画面外出たら消す
180          self.kill()
181
182  ✓ class Icicle(pg.sprite.Sprite):
183      """
184      上から落ちてくるつらら
185      """
186
187  ✓ def __init__(self):
188      super().__init__()
189
190      self.image = pg.Surface((20, 60))
191      self.image.fill((150, 255, 255))
192
193      self.rect = self.image.get_rect()
194
195      self.rect.x = random.randint(50, 330)
196      self.rect.y = 0
197
198      self.vy = random.randint(8, 15)
199
200  ✓ def update(self):
201      self.rect.y += self.vy
202
203      if self.rect.top > HEIGHT:
204          self.kill()
205
206  ✓ class Platform(pg.sprite.Sprite):
207      """
208      こうかとんが乗れる足場
209      """
210  ✓ def __init__(self, x, y, w=180, h=20):
211      super().__init__()
212
213      self.image = pg.Surface((w, h))
214      self.image.fill((139, 69, 19)) # 茶色
215
216      self.rect = self.image.get_rect()
217      self.rect.x = x
218      self.rect.y = y
219
220      self.vx = -6 # 背景に合わせて左へ移動
221
222  ✓ def update(self):
223      self.rect.x += self.vx
224
```

```
225         # 画面外に出たら消す
226         if self.rect.right < 0:
227             self.kill()
228     # class Score:
229     #     """
230     #     打ち落とした爆弾, 敵機の数スコアとして表示するクラス
231     #     爆弾: 1点
232     #     敵機: 10点
233     #     今からかえる
234     #     こうかとの座標を超えた障害物を判定
235     #     """
236     #     def __init__(self):
237     #         self.font = pg.font.Font(None, 50)
238     #         self.color = (0, 0, 255)
239     #         self.value = 0
240     #         self.image = self.font.render(f"Score: {self.value}", 0, self.color)
241     #         self.rect = self.image.get_rect()
242     #         self.rect.center = 100, HEIGHT-50
243
244     #     def update(self, screen: pg.Surface):
245     #         self.image = self.font.render(f"Score: {self.value}", 0, self.color)
246     #         screen.blit(self.image, self.rect)
247
248     # class Laser(pg.sprite.Sprite):
249     #     """
250     #     爆弾に関するクラス
251     #     今回はボスが出してくる攻撃などとして使用
252     #     """
253
254     #     def __init__(self, boss: "Boss", bird: Bird):
255     #         """
256     #         爆弾円Surfaceを生成する
257     #         引数1 emy: 爆弾を投下する敵機
258     #         引数2 bird: 攻撃対象のこうかとん
259     #         """
260     #         super().__init__()
261
262     #         # 爆弾を投下するemyから見た攻撃対象のbirdの方向を計算
263     #         self.vx, self.vy = calc_orientation(boss.rect, bird.rect)
264     #         self.rect.centerx = boss.rect.centerx
265     #         self.rect.centery = boss.rect.centery+boss.rect.height//2
266     #         self.speed = 6
267
268     #     def update(self):
269     #         """
270     #         爆弾を速度ベクトルself.vx, self.vyに基づき移動させる
271     #         引数 screen: 画面Surface
272     #         """
273     #         self.rect.move_ip(self.speed*self.vx, self.speed*self.vy)
274     #         if check_bound(self.rect) != (True, True):
275     #             self.kill()
276
277
278
279     # class Beam(pg.sprite.Sprite):
280     #     """
281     #     ビームに関するクラス
282     #     ボス倒すための攻撃手段
283     #     障害物に当たるとビームは消滅
284     #     """
```

```

285 #     def __init__(self, bird: Bird, angle0 = 0):         #課題6でangle0を追加
286 #         """
287 #         ビーム画像Surfaceを生成する
288 #         引数 bird: ビームを放つこうかとん
289 #         """
290 #         super().__init__()
291
292 #         self.vx, self.vy = bird.dire
293 #         angle = math.degrees(math.atan2(-self.vy, self.vx)) + angle0         #ビームの回転角度に加算
294 #         self.image = pg.transform.rotozoom(pg.image.load(f"fig/beam.png"), angle, 1.0)
295 #         self.vx = math.cos(math.radians(angle))
296 #         self.vy = -math.sin(math.radians(angle))
297 #         self.rect = self.image.get_rect()
298 #         self.rect.centery = bird.rect.centery+bird.rect.height*self.vy
299 #         self.rect.centerx = bird.rect.centerx+bird.rect.width*self.vx
300 #         self.speed = 10
301
302 #     def update(self):
303 #         """
304 #         ビームを速度ベクトルself.vx, self.vyに基づき移動させる
305 #         引数 screen: 画面Surface
306 #         """
307 #         self.rect.move_ip(self.speed*self.vx, self.speed*self.vy)
308 #         if check_bound(self.rect) != (True, True):
309 #             self.kill()
310
311
312 ▼ class Explosion(pg.sprite.Sprite):
313     """
314     爆発に関するクラス
315     今回はボスの放つ攻撃のための爆発
316     障害物も
317     """
318 ▼     def __init__(self, obj: "Obstacle", life: int):
319         """
320         爆弾が爆発するエフェクトを生成する
321         引数1 obj: 爆発するBombまたは敵機インスタンス Bomb|Enemy|が入ってた
322         引数2 life: 爆発時間
323         """
324         super().__init__()
325         img = pg.image.load(f"fig/explosion.gif")
326         self.imgs = [img, pg.transform.flip(img, 1, 1)]
327         self.image = self.imgs[0]
328         self.rect = self.image.get_rect(center=obj.rect.center)
329         self.life = life
330
331 ▼     def update(self):
332         """
333         爆発時間を1減算した爆発経過時間_lifeに応じて爆発画像を切り替えることで
334         爆発エフェクトを表現する
335         """
336         self.life -= 1
337         self.image = self.imgs[self.life//10%2]
338         if self.life < 0:
339             self.kill()
340
341
342 ▼ class Boss():
343     """
344     ボスのクラス

```



```

345     出現位置から降下させることで、ボス感を出している
346     インターバルはボスのレーザーの判定に使う（今回は違う）
347     """
348
349     def __init__(self):
350         super().__init__()
351         # self.image = pg.transform.rotozoom(random.choice(__class__.imgs), 0, 0.8)
352         self.image = pg.image.load("fig/alien1.png")
353         self.image = pg.transform.scale(self.image, (300, 300))
354         self.rect = self.image.get_rect()
355         self.rect.center = (900, 300)
356         self.vx, self.vy = 0, +6
357         self.bound = 330 # 停止位置
358         self.state = "down" # 降下状態or停止状態
359
360         self.interval = random.randint(50, 300) # 爆弾投下インターバル
361
362     def update(self, screen):
363         """
364         敵機を速度ベクトルself.vyに基づき移動（降下）させる
365         ランダムに決めた停止位置_boundまで降下したら、_stateを停止状態に変更する
366         引数 screen：画面Surface
367         """
368         if self.rect.centery > self.bound:
369             self.vy = 0
370             self.state = "stop"
371         self.rect.move_ip(self.vx, self.vy)
372
373         screen.blit(self.image, self.rect)
374
375
376     class Bomb(pg.sprite.Sprite):
377         """
378         ボスの攻撃の一つ
379         爆弾円を生成し、爆破場所を知らせたのち、爆発画像に切り替える
380         衝突判定は爆発画像から
381         生成場所は、こうかとんが動ける範囲の中でランダムな場所
382         （これは今回はマージできてないので使用できない）
383         爆発中かどうかのステータス：bom_status
384         により衝突判定を切り替える
385         爆発までの時間：bom_timer
386         """
387     def __init__(self, boss: "Boss", bird: Bird, triple = 0):
388         """
389         引数：tripleは、爆弾を三つ生成する際に使用するだけのため
390         初期値を0とし、爆弾を一つ生成する場合には影響を与えない
391         """
392         super().__init__()
393         rad = 40 # 爆弾円の半径
394         self.image = pg.Surface((2*rad, 2*rad))
395         color = (255, 0, 0) # 爆弾円の色
396         pg.draw.circle(self.image, color, (rad, rad), rad)
397         self.image.set_colorkey((0, 0, 0))
398         self.rect = self.image.get_rect()
399
400         self.rect.center = bird.rect.center
401
402         #こうかとんが動ける範囲をランダムに爆発する
403         self.rect.center = (
404             random.randint(50, 350) + triple,

```

```

405         random.randint(50, GROUND + 200)
406     )
407
408     self.bom_timer = 0
409     self.bom_status = "normal"
410
411     def update(self):
412         #爆弾円が表示されて、数秒経ったら爆発画像に切り替える
413
414         self.bom_timer += 1
415         if self.bom_timer >= 200 and self.bom_status == "normal":
416
417             center = self.rect.center
418
419             self.image = pg.image.load(f"fig/explosion.gif")
420             self.rect = self.image.get_rect()
421             self.rect.center = center
422
423             self.bom_status = "explosion"
424
425             if self.bom_timer >= 250:
426                 self.bom_status == "normal"
427                 self.kill()
428
429
430     class Life():
431         """
432         体力
433         障害物または敵の攻撃などに当たると減る
434         """
435     def __init__(self, num):
436         self.num = num
437         self.img = pg.Surface((40, 40))
438         points = [(16*math.sin(t/100)*3 +20,
439                 -(13*math.cos(t/100)-5*math.cos(2*t/100)-2*math.cos(3*t/100)-math.cos(4*t/100)) +20
440                 ) for t in range(0, 628) ]
441         pg.draw.polygon(self.img, (255, 0, 0), points)
442         self.img.set_colorkey((0, 0, 0))
443
444     def update(self, screen):
445         for i in range(self.num):
446             x = WIDTH - 50 - i * 45
447             y = HEIGHT - 600
448             screen.blit(self.img, (x, y))
449
450
451     def main():
452         pg.display.set_caption("走れ！こうかとん")
453         screen = pg.display.set_mode((1100, 650))
454         clock = pg.time.Clock()
455         move = 0
456         tmr = 0
457         maps = Map()    #マップを切り替えるため
458         x = 0 #練習5
459
460
461         # score = Score()
462         bird = Bird()
463         boss = Boss()
464

```

```
465     bombs = pg.sprite.Group()
466     # beams = pg.sprite.Group()
467     exps = pg.sprite.Group()
468     # emys = pg.sprite.Group()
469     obstacle = pg.sprite.Group()
470     platforms = pg.sprite.Group()
471     icicles = pg.sprite.Group()
472
473     life = Life(5)
474     attack_count = 0
475     bomb_cooldown = 0
476     # gravity = pg.sprite.Group()      #課題2 Groupにインスタンスを追加
477     # shield = pg.sprite.Group()
478     while True:
479
480         bomb_cooldown += 1
481
482
483         if tmr >= 6100:
484
485             #爆弾が画面上になければ、生成する
486             if len(bombs) == 0 and bomb_cooldown >= 200 and attack_count < 3:
487                 bombs.add(Bomb(boss, bird))
488                 bomb_cooldown = 0
489                 attack_count += 1
490
491             #bombの処理 爆発を三回させるかどうか
492             if len(bombs) == 0 and attack_count >= 3:
493                 #重なってしまうので、ずれた場所に生成する
494                 bombs.add(Bomb(boss, bird, -50))
495                 bombs.add(Bomb(boss, bird, 0))
496                 bombs.add(Bomb(boss, bird, 50))
497                 attack_count = 0
498
499             #障害物
500             if tmr % 60 == 0:
501                 obstacle.add(Obstacle())
502             # タフステージだけつら生成
503             if 2000 <= tmr < 4000 and tmr % 80 == 0:
504                 icicles.add(Icicle())
505                 if tmr % 80 == 0:
506                     icicles.add(Icicle())
507             # 足場生成
508             if tmr % 180 == 0:
509                 y = random.randint(250, 450)
510                 platforms.add(Platform(WIDTH, y))
511
512             #ステージ名表示のため一時停止
513             if tmr == 0:
514                 bg1_img = pg.image.load("fig/pg2_bg.png")
515                 screen.blit(bg1_img, (0, 0))
516
517                 title_shade = pg.Surface((430, 150))
518                 title_shade.fill((0, 0, 0))
519                 title_shade.set_alpha(180)
520                 screen.blit(title_shade, (350, 250))
521                 font = pg.font.Font(None, 80)
522                 txt = font.render("Morning stage", True, (0, 255, 255))
523                 screen.blit(txt, (400, 300))
524                 pg.display.update()
```

```
525         time.sleep(3)
526
527     if tmr == 2000:
528         #初期設定に戻す
529         bird.rect.x = 50
530         bird.rect.y = GROUND + 140
531         bird.vy = 0
532         bird.jumping = False
533
534         #障害物も消す
535         obstacle.empty()
536         #背景がステージコールの時に反映されないからここで
537         screen.blit(maps.bg2_img, (0, 0))
538
539         title_shade = pg.Surface((430, 150))
540         title_shade.fill((0, 0, 0))
541         title_shade.set_alpha(180)
542         screen.blit(title_shade, (350, 250))
543         font = pg.font.Font(None, 80)
544         txt = font.render("Noon stage", True, (0, 255, 255))
545         screen.blit(txt, (400, 300))
546         pg.display.update()
547         time.sleep(3)
548
549     if tmr == 4000:
550         #初期設定に戻す
551         bird.rect.x = 50
552         bird.rect.y = GROUND + 140
553         bird.vy = 0
554         bird.jumping = False
555
556         #障害物も消す
557         obstacle.empty()
558         #背景がステージコールの時に反映されないからここで
559         screen.blit(maps.bg3_img, (0, 0))
560
561         title_shade = pg.Surface((430, 150))
562         title_shade.fill((0, 0, 0))
563         title_shade.set_alpha(180)
564         screen.blit(title_shade, (350, 250))
565         font = pg.font.Font(None, 80)
566         txt = font.render("Night stage", True, (0, 255, 255))
567         screen.blit(txt, (400, 300))
568         pg.display.update()
569         time.sleep(3)
570
571
572     key_lst = pg.key.get_pressed()
573     for event in pg.event.get():
574         if event.type == pg.QUIT: return
575
576         if event.type == pg.KEYDOWN and event.key == pg.K_SPACE:
577             # 2回までジャンプ可能
578             if bird.jump_count < bird.max_jump:
579                 bird.vy = -15
580                 bird.jumping = True
581                 bird.jump_count += 1
582
583
584
```

```
585     # for bomb in pg.sprite.spritecollide(bird, bombs, True): # こうかたと衝突した爆弾リスト
586     #     if bird.state == "hyper": #課題4の無敵時間判定のために条件分岐
587     #         exps.add(Explosion(bomb, 50))
588     #         score.value += 1
589     #     else:
590     #         bird.change_img(8, screen) # こうかたとん悲しみエフェクト
591     #         score.update(screen)
592     #         life.num -= 1 #課題1
593     #         pg.display.update()
594     #         if life.num <= 0:
595     #             time.sleep(2)
596     #             return
597
598     # if event.type == pg.KEYDOWN and event.key == pg.K_RETURN: #課題2の重力場判定
599     #     if score.value >= 10: #scoreクラスの中にvalueという名前で書かれていたため
600     #         gravity.add(Gravity(400))
601     #         score.value -= 10 #見せるために一時的に10にしているだけだからあとで200に変えて
602
603     # if event.type == pg.KEYDOWN and event.key == pg.K_e: #課題3
604     #     if score.value >= 20:
605     #         EMP(emys, bombs, screen)
606     #         score.value -= 20
607
608     # if event.type == pg.KEYDOWN and event.key == pg.K_RSHIFT: #課題4
609     #     if score.value >= 10: #これも課題のため10にしておくけど、100に戻しておいて
610     #         bird.state = "hyper"
611     #         bird.hyper_life = 500
612     #         score.value -= 10
613
614     # if event.type == pg.KEYDOWN and event.key == pg.K_s: #課題5
615     #     if score.value >= 0 and len(shield) == 0:
616     #         shield.add(Shield(400, bird))
617     #         score.value -= 0
618
619     # if tmr%200 == 0: # 200フレームに1回、敵機を出現させる
620     #     emys.add(Enemy())
621
622     # if boss.state == "stop" and tmr%boss.interval == 0:
623     #     # 敵機が停止状態に入ったら、intervalに応じて爆弾投下
624     #     bombs.add(Bomb(boss, bird))
625
626
627     # for emy in pg.sprite.groupcollide(emys, beams, True, True).keys(): # ビームと衝突した敵機リスト
628     #     exps.add(Explosion(emy, 100)) # 爆発エフェクト
629     #     score.value += 10 # 10点アップ
630     #     bird.change_img(6, screen) # こうかたとん喜びエフェクト
631
632     # for bomb in pg.sprite.groupcollide(bombs, beams, True, True).keys(): # ビームと衝突した爆弾リスト
633     #     exps.add(Explosion(bomb, 50)) # 爆発エフェクト
634     #     score.value += 1 # 1点アップ
635
636     # for bomb in pg.sprite.spritecollide(bird, bombs, True): # こうかたと衝突した爆弾リスト
637     #     if bird.state == "hyper": #課題4の無敵時間判定のために条件分岐
638     #         exps.add(Explosion(bomb, 50))
639     #         score.value += 1
640     #     else:
641     #         bird.change_img(8, screen) # こうかたとん悲しみエフェクト
642     #         score.update(screen)
643     #         life.num -= 1 #課題1
644     #         pg.display.update()
```

```
645         #         if life.num <= 0:
646         #             time.sleep(2)
647         #             return
648
649         # for emy in pg.sprite.groupcollide(emy, gravity, True, False).keys(): # 重力場と敵機の衝突判定
650         #     exps.add(Explosion(emy, 100)) # 爆発エフェクト
651
652         # for bomb in pg.sprite.groupcollide(bombs, gravity, True, False):
653         #     exps.add(Explosion(bomb, 50))
654
655         #グループからbombを取り出して、bom_statusが
656         # "explosion"の時だけ衝突判定させる
657         for bomb in bombs:
658             if bomb.bom_status == "explosion":
659                 for bomb in pg.sprite.spritecollide(bird, bombs, True):
660                     exps.add(Explosion(bomb, 50))
661                     life.num -= 1
662                     pg.display.update()
663                     if life.num <= 0:
664                         time.sleep(2)
665                         return
666
667         for obstacles in pg.sprite.spritecollide(bird, obstacle, True):
668             exps.add(Explosion(obstacles, 50))
669             #こうかとん悲しみエフェクト
670             # bird.change_img(8, screen)
671             # score.update(screen)
672             life.num -= 1 #ここでlifeを減らす
673             pg.display.update()
674             if life.num <= 0:
675                 time.sleep(2)
676                 return
677
678         # つららとの当たり判定
679         for icicle in pg.sprite.spritecollide(bird, icicles, True):
680             exps.add(Explosion(icicle, 50))
681             life.num -= 1
682             pg.display.update()
683
684             if life.num <= 0:
685                 time.sleep(2)
686                 return
687         maps.update(screen, tmr)
688
689         bird.update(screen)
690         if tmr >= 6000:
691             boss.update(screen)
692         bird.update(screen, platforms)
693
694         # beams.update()
695         # beams.draw(screen)
696         # emys.update()
697         # emys.draw(screen)
698         # bombs.update()
699         # bombs.draw(screen)
700         exps.update()
701         exps.draw(screen)
702         # score.update(screen)
703         life.update(screen)
704
```

```
705     platforms.update()
706     platforms.draw(screen)
707
708     obstacle.update()
709     obstacle.draw(screen)
710     icicles.update()
711     icicles.draw(screen)
712
713     bombs.update()
714     bombs.draw(screen)
715
716     pg.display.update()
717     tmr += 1
718     clock.tick(50) #FPSはこれ フレーム数。50フレームで1秒を表すということ
719
720
721 if __name__ == "__main__":
722     pg.init()
723     main()
724     pg.quit()
725     sys.exit()
```

c0a2515415 / ProjExD_Group07

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

Insights

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base: 18ca61060c4faeca11f4ebea439...

compare: fdbbcc412a7c846ab9e025cff1d...

3 commits

1 file changed

1 contributor

Commits on May 19, 2026

- ボス実装と攻撃実装は完了

c0a2515415 committed 1 hour ago

9c144fc

<>

ボス機能実装完了

c0a2515415 committed 51 minutes ago

51e33bc

<>

ボス機能最終実装完了

c0a2515415 committed 16 minutes ago

fdbbcc4

<>

Showing 1 changed file with 134 additions and 105 deletions.

Split

Unified

239 rush_kokaton.py			
164	# self.image = self.font.render(f"Score: {self.value}", 0, self.color)	164	# self.image = self.font.render(f"Score: {self.value}", 0, self.color)
165	# screen.blit(self.image, self.rect)	165	# screen.blit(self.image, self.rect)
166		166	
167	-	167	+ # class Laser(pg.sprite.Sprite):
168	- # class Bomb(pg.sprite.Sprite):		
169	# """	168	# """
170	# 爆弾に関するクラス	169	# 爆弾に関するクラス
171	# 今回はボスが出してくる攻撃などとして使用	170	# 今回はボスが出してくる攻撃などとして使用
172	# """	171	# """
173	- # colors = [(255, 0, 0), (0, 255, 0), (0, 0, 255), (255, 255, 0), (255, 0, 255), (0, 255, 255)]		
174		172	
175	- # def __init__(self, emy: "Enemy", bird: Bird):	173	+ # def __init__(self, boss: "Boss", bird: Bird):
176	# """	174	# """
177	# 爆弾円Surfaceを生成する	175	# 爆弾円Surfaceを生成する
178	# 引数1 emy: 爆弾を投下する敵機	176	# 引数1 emy: 爆弾を投下する敵機
179	# 引数2 bird: 攻撃対象のこうかとん	177	# 引数2 bird: 攻撃対象のこうかとん
180	# """	178	# """
181	# super().__init__()	179	# super().__init__()
182	- # rad = random.randint(10, 50) # 爆弾 円の半径: 10以上50以下の乱数	180	+

https://github.com/c0a2515415/ProjExD_Group07/compare/18ca61060c4faeca11f4ebea4394fde9323272f5...fdbbcc412a7c846ab9e025cff1d27f6... 1/7

183	- # self.image = pg.Surface((2*rad, 2*rad))		
184	- # color = random.choice(__class__.colors) # 爆弾円の色 : クラス変数からランダム選択		
185	- # pg.draw.circle(self.image, color, (rad, rad), rad)		
186	- # self.image.set_colorkey((0, 0, 0))		
187	- # self.rect = self.image.get_rect()		
188	# # 爆弾を投下するemyから見た攻撃対象の birdの方向を計算	181	# # 爆弾を投下するemyから見た攻撃対象の birdの方向を計算
189	- # self.vx, self.vy = calc_orientation(emy.rect, bird.rect)	182	+ # self.vx, self.vy = calc_orientation(boss.rect, bird.rect)
190	- # self.rect.centerx = emy.rect.centerx	183	+ # self.rect.centerx = boss.rect.centerx
191	- # self.rect.centery = emy.rect.centery+emy.rect.height//2	184	+ # self.rect.centery = boss.rect.centery+boss.rect.height//2
192	# self.speed = 6	185	# self.speed = 6
193		186	
194	# def update(self):	187	# def update(self):
201	# self.kill()	194	# self.kill()
202		195	
203		196	
		197	+
204	# class Beam(pg.sprite.Sprite):	198	# class Beam(pg.sprite.Sprite):
205	# """	199	# """
206	# ビームに関するクラス	200	# ビームに関するクラス
264	self.kill()	258	self.kill()
265		259	
266		260	
267	- # class Enemy(pg.sprite.Sprite):	261	+ class Boss():
268	- # """	262	+ """
269	- # 敵機に関するクラス	263	+ ボスのクラス
270	- # ボスのクラス (大きさだけいじればいいかな)	264	+ 出現位置から降下させることで、ボス感を出している
271	- # """	265	+ インターバルはボスのレーザーの判定に使う (今回は違う)
272	- # imgs = [pg.image.load(f"fig/alien{i}.png") for i in range(1, 4)]	266	+ """
273		267	
274	- # def __init__(self):	268	+ def __init__(self):
275	- # super().__init__()	269	+ super().__init__()
276	- # self.image = pg.transform.rotozoom(random.choice(__class__.imgs), 0, 0.8)	270	+ # self.image = pg.transform.rotozoom(random.choice(__class__.imgs), 0, 0.8)
277	- # self.rect = self.image.get_rect()	271	+ self.image = pg.image.load("fig/alien1.png")
278	- # self.rect.center = random.randint(0, WIDTH), 0	272	+ self.image = pg.transform.scale(self.image, (300, 300))
279	- # self.vx, self.vy = 0, +6	273	+ self.rect = self.image.get_rect()
280	- # self.bound = random.randint(50, HEIGHT//2) # 停止位置	274	+ self.rect.center = (900, 300)
281	- # self.state = "down" # 降下状態or停止状態	275	+ self.vx, self.vy = 0, +6
282	- # self.interval = random.randint(50, 300) # 爆弾投下インターバル	276	+ self.bound = 330 # 停止位置

		277	+	<code>self.state = "down"</code> # 降下状態or停止状態
283		278		
284	- # <code>def update(self):</code>	279	+	<code>self.interval = random.randint(50,</code>
285	- # <code>"""</code>	280	+	<code>300)</code> # 爆弾投下インターバル
286	- # 敵機を速度ベクトル <code>self.vy</code> に基づき移動 (降下) させる	281	+	<code>def update(self, screen):</code>
287	- # ランダムに決めた停止位置 <code>_bound</code> まで降下 したら, <code>_state</code> を停止状態に変更する	282	+	<code>"""</code>
288	- # 引数 <code>screen</code> : 画面Surface	283	+	敵機を速度ベクトル <code>self.vy</code> に基づき移動 (降 下) させる
289	- # <code>"""</code>	284	+	ランダムに決めた停止位置 <code>_bound</code> まで降下し たら, <code>_state</code> を停止状態に変更する
290	- # <code>if self.rect.centery > self.bound:</code>	285	+	引数 <code>screen</code> : 画面Surface
291	- # <code>self.vy = 0</code>	286	+	<code>"""</code>
292	- # <code>self.state = "stop"</code>	287	+	<code>if self.rect.centery > self.bound:</code>
293	- # <code>self.rect.move_ip(self.vx, self.vy)</code>	288	+	<code>self.vy = 0</code>
		289	+	<code>self.state = "stop"</code>
		290	+	<code>self.rect.move_ip(self.vx, self.vy)</code>
		291	+	
		292	+	<code>screen.blit(self.image, self.rect)</code>
		293	+	
		294	+	
		295	+	<code>class Bomb(pg.sprite.Sprite):</code>
		296	+	<code>"""</code>
		297	+	ボスの攻撃の一つ
		298	+	爆弾円を生成し、爆破場所を知らせたのち、爆発画 像に切り替える
		299	+	衝突判定は爆発画像から
		300	+	生成場所は、こうかとんが動ける範囲の中でランダ ムな場所
		301	+	(これは今回はマージできてないので使用できな い)
		302	+	爆発中かどうかのステータス: <code>bom_status</code>
		303	+	により衝突判定を切り替える
		304	+	爆発までの時間: <code>bom_timer</code>
		305	+	<code>"""</code>
		306	+	<code>def __init__(self, boss: "Boss", bird:</code> <code>Bird, triple = 0):</code>
		307	+	<code>"""</code>
		308	+	引数: <code>triple</code> は、爆弾を三つ生成する際に使用 するだけのため
		309	+	初期値を0とし、爆弾を一つ生成する場合には 影響を与えない
		310	+	<code>"""</code>
		311	+	<code>super().__init__()</code>
		312	+	<code>rad = 40</code> # 爆弾円の半径
		313	+	<code>self.image = pg.Surface((2*rad,</code> <code>2*rad))</code>
		314	+	<code>color = (255, 0, 0)</code> # 爆弾円の色
		315	+	<code>pg.draw.circle(self.image, color,</code> <code>(rad, rad), rad)</code>
		316	+	<code>self.image.set_colorkey((0, 0, 0))</code>
		317	+	<code>self.rect = self.image.get_rect()</code>
		318	+	
		319	+	<code>self.rect.center = bird.rect.center</code>
		320	+	

		<pre> 321 + #こうかとんが動ける範囲をランダムに爆発す る 322 + self.rect.center = (323 + random.randint(50, 350) + triple, 324 + random.randint(50, GROUND + 200) 325 +) 326 + 327 + self.bom_timer = 0 328 + self.bom_status = "normal" 329 + 330 + def update(self): 331 + #爆弾円が表示されて、数秒経ったら爆発画像 に切り替える 332 + 333 + self.bom_timer += 1 334 + if self.bom_timer >= 200 and self.bom_status == "normal": 335 + 336 + center = self.rect.center 337 + 338 + self.image = pg.image.load(f"fig/explosion.gif") 339 + self.rect = self.image.get_rect() 340 + self.rect.center = center 341 + 342 + self.bom_status = "explosion" 343 + 344 + if self.bom_timer >= 250: 345 + self.bom_status == "normal" 346 + self.kill() </pre>	
294		347	
295		348	
296	<pre>class Life():</pre>	349	<pre>class Life():</pre>
314	<pre> screen.blit(self.img, (x, y))</pre>	367	<pre> screen.blit(self.img, (x, y))</pre>
315		368	
316		369	
317	<pre>- # class Gravity(pg.sprite.Sprite):</pre>		
318	<pre>- # """</pre>		
319	<pre>- # 今回は、小さい敵と障害物と爆弾を消せる必殺</pre>		
	<pre> 技クラス</pre>		
320	<pre>- # """</pre>		
321	<pre>- # def __init__(self, life):</pre>		
322	<pre>- # super().__init__() #Groupのため</pre>		
	<pre> に継承</pre>		
323	<pre>-</pre>		
324	<pre>- # self.life = life</pre>		
325	<pre>- # self.image = pg.Surface((WIDTH,</pre>		
	<pre> HEIGHT))</pre>		
326	<pre>- # self.image.set_alpha(128)</pre>		
327	<pre>- # self.rect = self.image.get_rect()</pre>		
328	<pre>-</pre>		
329	<pre>- # def update(self):</pre>		
330	<pre>- # self.life -= 1</pre>		
331	<pre>- # if self.life <= 0:</pre>		
332	<pre>- # self.kill()</pre>		
333	<pre>-</pre>		
334	<pre>-</pre>		
335	<pre>- # class EMP():</pre>		

```

336 - #     def __init__(self, emys, bombs,
      screen):
337 - #         for emy in emys:
338 - #             emy.interval = float("inf")
339 - #             emy.image =
      pg.transform.laplacian(emy.image)
340 -
341 - #         for bom in bombs:
342 - #             bom.speed /= 2
343 - #             bom.state = "inactive"
344 -
345 - #         image = pg.Surface((WIDTH, HEIGHT))
346 - #         image.fill((255, 255, 0))
347 - #         image.set_alpha(200)
348 -
349 - #         screen.blit(image, [0, 0])
350 - #         pg.display.update()
351 - #         time.sleep(0.05)
352 -
353 -
354 - # class Shield(pg.sprite.Sprite):
355 - #     def __init__(self, life, bird:Bird):
356 - #         super().__init__()
357 - #         self.life = life
358 -
359 - #         self.image = pg.Surface((20,
      bird.rect.height * 2))
360 - #         pg.draw.rect(self.image, (0, 0,
      255), (0, 0, 10, bird.rect.height * 2))
361 - #         self.rect = self.image.get_rect()
362 -
363 - #         vx, vy = bird.dire #向いてる方向のベ
      クトルをvx, vyに代入
364 - #         self.rect.centerx =
      bird.rect.centerx + vx * bird.rect.width
365 - #         self.rect.centery =
      bird.rect.centery + vy * bird.rect.height
366 - #         kakudo = math.degrees(math.atan2(-
      vy, vx))
367 - #         self.image =
      pg.transform.rotozoom(self.image, kakudo, 1)
368 - #         self.image.set_colorkey((0, 0, 0))
      #ここに入れないと黒くなる
369 -
370 - #     def update(self):
371 - #         self.life -= 1
372 - #         if self.life <= 0:
373 - #             self.kill()
374 -
375 -

```

```

376 def main():
377     pg.display.set_caption("走れ！こうかとん")
378     screen = pg.display.set_mode((1100, 650))
385
386     # score = Score()
387     bird = Bird()

```

388

```

370 def main():
371     pg.display.set_caption("走れ！こうかとん")
372     screen = pg.display.set_mode((1100, 650))
379
380     # score = Score()
381     bird = Bird()
382 + boss = Boss()

```

383

389	-	# bombs = pg.sprite.Group()	384	+	bombs = pg.sprite.Group()
390		# beams = pg.sprite.Group()	385		# beams = pg.sprite.Group()
391		exps = pg.sprite.Group()	386		exps = pg.sprite.Group()
392		# emys = pg.sprite.Group()	387		# emys = pg.sprite.Group()
393		obstacle = pg.sprite.Group()	388		obstacle = pg.sprite.Group()
394			389		
395		life = Life(5)	390		life = Life(5)
			391	+	attack_count = 0
			392	+	bomb_cooldown = 0
396		# gravity = pg.sprite.Group() #課題2 Groupにインスタンスを追加	393		# gravity = pg.sprite.Group() #課題2 Groupにインスタンスを追加
397		# shield = pg.sprite.Group()	394		# shield = pg.sprite.Group()
398		while True:	395		while True:
399			396		
			397	+	bomb_cooldown += 1
			398	+	
			399	+	
			400	+	if tmr >= 6100:
			401	+	
			402	+	#爆弾が画面上になければ、生成する
			403	+	if len(bombs) == 0 and
					bomb_cooldown >= 200 and attack_count < 3:
			404	+	bombs.add(Bomb(boss, bird))
			405	+	bomb_cooldown = 0
			406	+	attack_count += 1
			407	+	
			408	+	#bombの処理 爆発を三回させるかどうか
			409	+	if len(bombs) == 0 and
					attack_count >= 3:
			410	+	#重なってしまうので、ずれた場所に生成する
			411	+	bombs.add(Bomb(boss, bird,
					-50))
			412	+	bombs.add(Bomb(boss, bird,
					0))
			413	+	bombs.add(Bomb(boss, bird,
					50))
			414	+	attack_count = 0
400			415		
401		#障害物	416		#障害物
402		if tmr % 60 == 0:	417		if tmr % 60 == 0:
506		# if tmr%200 == 0: # 200フレームに1回, 敵機を出現させる	521		# if tmr%200 == 0: # 200フレームに1回, 敵機を出現させる
507		# emys.add(Enemy())	522		# emys.add(Enemy())
508			523		
509	-	# for emy in emys:	524	+	# if boss.state == "stop" and
					tmr%boss.interval == 0:
510	-	# if emy.state == "stop" and	525	+	# # 敵機が停止状態に入ったら,
		tmr%emy.interval == 0:			intervalに応じて爆弾投下
511	-	# # 敵機が停止状態に入ったら,	526	+	# bombs.add(Bomb(boss, bird))
		intervalに応じて爆弾投下			
512	-	# bombs.add(Bomb(emy, bird))	527	+	
513			528		
514		# for emy in	529		# for emy in
		pg.sprite.groupcollide(emys, beams, True,			pg.sprite.groupcollide(emys, beams, True,
		True).keys(): # ビームと衝突した敵機リスト			True).keys(): # ビームと衝突した敵機リスト

515	# exps.add(Explosion(emy, 100))	530	# exps.add(Explosion(emy, 100))
	# 爆発エフェクト		# 爆発エフェクト
539	# for bomb in	554	# for bomb in
	pg.sprite.groupcollide(bombs, gravity, True, False):		pg.sprite.groupcollide(bombs, gravity, True, False):
540	# exps.add(Explosion(bomb, 50))	555	# exps.add(Explosion(bomb, 50))
541		556	
542	- # for bomb in	557	+ #グループからbombを取り出して、 bom_statusが
	pg.sprite.groupcollide(shield, bombs, True, True): #課題5 防御壁		
543	- # exps.add(Explosion(bomb, 50))	558	+ #"explosion"の時だけ衝突判定させる
544	-	559	+ for bomb in bombs:
		560	+ if bomb.bom_status ==
			"explosion":
		561	+ for bomb in
			pg.sprite.spritecollide(bird, bombs, True):
		562	+ exps.add(Explosion(bomb,
			50))
		563	+ life.num -= 1
		564	+ pg.display.update()
		565	+ if life.num <= 0:
		566	+ time.sleep(2)
		567	+ return
		568	+
545	for obstacles in	569	for obstacles in
	pg.sprite.spritecollide(bird, obstacle, True):		pg.sprite.spritecollide(bird, obstacle, True):
546	exps.add(Explosion(obstacles,	570	exps.add(Explosion(obstacles,
	50))		50))
547	#こうかтон悲しみエフェクト	571	#こうかтон悲しみエフェクト
557	maps.update(screen, tmr)	581	maps.update(screen, tmr)
558		582	
559	bird.update(screen)	583	bird.update(screen)
		584	+ if tmr >= 6000:
		585	+ boss.update(screen)
560		586	
561	# beams.update()	587	# beams.update()
562	# beams.draw(screen)	588	# beams.draw(screen)
572	obstacle.update()	598	obstacle.update()
573	obstacle.draw(screen)	599	obstacle.draw(screen)
574		600	
		601	+ bombs.update()
		602	+ bombs.draw(screen)
		603	+
575	pg.display.update()	604	pg.display.update()
576	tmr += 1	605	tmr += 1
577	clock.tick(50) #FPSはこれ フレーム数。	606	clock.tick(50) #FPSはこれ フレーム数。
	50フレームで1秒を表すということ		50フレームで1秒を表すということ